Roteiro da apresentação

Sistemas Operacionais - Atividade 3 - UFS 2026.2

Duração-alvo: 11 minutos, com perguntas separadas de 3-5 minutos.

Texto em FALA é a sugestão de locução. EM TELA orienta o operador. Tempos são estimativas para ensaio, não medições de uma gravação. Todos devem compreender sua parte. A divisão de falas não comprova contribuição técnica efetiva.

Sequência dos 11 slides

Slide e assunto	Quem fala	Janela
01. Virtualização, namespaces e cgroups	Bruno	00:00-00:40
02. Ambiente e kernel compartilhado	Iasmin	00:40-01:40
03. Evidências de namespaces	Pedro	01:40-02:50
04. Carga determinística e protocolo	Christian	02:50-04:00
05. Limites efetivos do cgroup	Marcos	04:00-05:00
06. Seis medições de tempo decorrido	Sibele	05:00-06:20
07. Throttling e escopos de memória	Marcos + Sibele	06:20-07:20
08. Mount read-only e menor privilégio	Silas	07:20-08:30
09. Evidência selecionada	Christian	08:30-09:30
10. Limitações e relação com inferência	Iasmin + Silas	09:30-10:30
11. Equipe e revisão final	Bruno	10:30-11:00

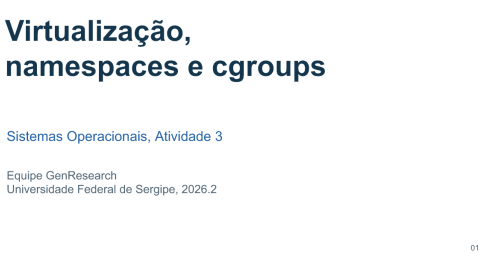
Participação por integrante

Integrante	Slides	Tempo-alvo
Bruno Amancio Ferreira	1, 11	01:10
Christian Will Silva Santos Nunes	4, 9	02:10
Iasmin Victoria Teixeira Barreto	2, 10	01:30
Marcos Vinícius Dantas Aguiar	5, 7	01:30
Pedro César Figueiredo Carneiro	3	01:10
Sibele Oliveira Cruz	6, 7	01:50
Silas Santos da Silva	8, 10	01:40

Base: apresentacao_atividade3.pptx, 11 slides. O slide 11 ainda contém proposta de participação e pendências; atualizar com informações confirmadas antes da versão final. As instruções de operação e preparação aparecem no final deste PDF.

Slide 01 - Virtualização, namespaces e cgroups

Janela 00:00-00:40 | Duração-alvo 00:40



EM TELA - não ler em voz alta

Manter a capa e olhar para a turma. Apresentar o objetivo antes dos detalhes técnicos.

Próxima ação

Trocar para o slide 2 quando Bruno anunciar Iasmin.

FALA - Bruno Amancio Ferreira (00:00-00:40)

Olá, nós somos a equipe GenResearch, e eu sou Bruno. Nesta atividade de Sistemas Operacionais, investigamos como contêineres Linux isolam a visão dos processos e limitam recursos usando o mesmo kernel. Criamos um laboratório próprio, com dois contêineres e seis medições de uma carga de CPU. Vamos apresentar o ambiente, as evidências de isolamento, os resultados e suas limitações. Iasmin começa mostrando onde esses contêineres executam.

Slide 02 - Ambiente e kernel compartilhado

Janela 00:40-01:40 | Duração-alvo 01:00



EM TELA - não ler em voz alta

Apontar o diagrama de cima para baixo: Windows, WSL2, kernel, daemon e contêineres.

Próxima ação

Trocar para o slide 3 após a passagem para Pedro.

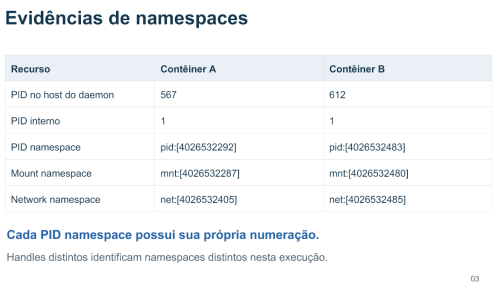
FALA - Iasmin Victoria Teixeira Barreto (00:40-01:40)

Eu sou Iasmin. O host físico utiliza Windows 10. Acima dele, a infraestrutura WSL2 fornece o ambiente Linux em que executa o Docker Desktop. É o kernel desse ambiente Linux que os contêineres compartilham. Cada imagem fornece seu espaço de usuário, com Python e bibliotecas Debian.

O daemon anunciou doze CPUs lógicas e aproximadamente 3,72 gibibytes de RAM. Esses valores descrevem a camada Linux intermediária. O Ubuntu usado na análise também precisa ser distinguido do ambiente efetivo do daemon. Neste desenho existe uma camada de virtualização, mas cada contêiner continua sendo um conjunto de processos sob o kernel compartilhado. Pedro vai mostrar como esses processos podem ter visões diferentes.

Slide 03 - Evidências de namespaces

Janela 01:40-02:50 | Duração-alvo 01:10



Evidências de namespaces

Recurso	Contêiner A	Contêiner B
PID no host do daemon	567	612
PID interno	1	1
PID namespace	pid:[4026532292]	pid:[4026532483]
Mount namespace	mnt:[4026532287]	mnt:[4026532480]
Network namespace	net:[4026532405]	net:[4026532485]

Cada PID namespace possui sua própria numeração.

Handles distintos identificam namespaces distintos nesta execução.

03

EM TELA - não ler em voz alta

Apontar primeiro os PIDs 567 e 612, depois os dois PIDs internos iguais a 1 e as três linhas de handles.

Próxima ação

Trocar para o slide 4 na passagem para Christian.

FALA - Pedro César Figueiredo Carneiro (01:40-02:50)

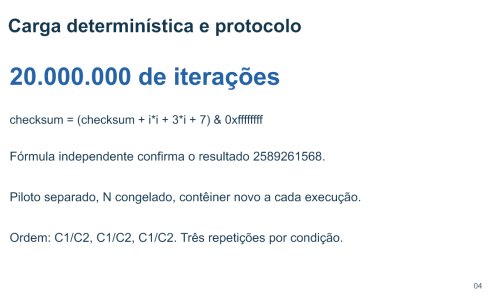
Eu sou Pedro. Coletamos os dados enquanto dois contêineres independentes estavam ativos. O Docker informou os PIDs 567 e 612 no host do daemon. Dentro de cada contêiner, o processo inicial tinha PID um. Isso acontece porque a numeração é relativa ao PID namespace de cada contêiner.

Na tabela, os identificadores de PID, mount e network são diferentes entre A e B. Esses handles sustentam que os contêineres pertenciam a namespaces distintos. O namespace de PID organiza a visão dos processos; o de mount, a visão das montagens; e o de network, os recursos de rede.

A diferença dos identificadores não comprova isolamento absoluto. A rede estava desativada, e não fizemos um teste de tráfego. Christian vai explicar o trabalho usado para comparar as quotas de CPU.

Slide 04 - Carga determinística e protocolo

Janela 02:50-04:00 | Duração-alvo 01:10



EM TELA - não ler em voz alta

Apontar vinte milhões, o checksum e a ordem C1/C2. A fórmula do slide é uma operação de acumulação, não inferência de IA.

Próxima ação

Trocar para o slide 5 na passagem para Marcos.

FALA - Christian Will Silva Santos Nunes (02:50-04:00)

Eu sou Christian. Para comparar as quotas, usamos uma carga de CPU com uma única thread e vinte milhões de iterações. O laço acumula uma expressão aritmética e limita o resultado a trinta e dois bits. Uma fórmula fechada calcula o checksum esperado de forma independente, e as seis execuções chegaram ao mesmo resultado indicado no slide.

Primeiro fizemos um piloto separado, com um milhão de iterações. Depois congelamos o trabalho principal e mantivemos a mesma quantidade nas seis medições. Cada execução usou um contêiner novo, alternando C1 e C2 três vezes.

Assim, a comparação mantém o trabalho e a imagem, variando a quota de CPU. O piloto não entra nas estatísticas. A carga não utiliza LLM, GPU ou espera simulada. Marcos vai mostrar os limites que realmente observamos no kernel.

Slide 05 - Limites efetivos do cgroup

Janela 04:00-05:00 | Duração-alvo 01:00



EM TELA - não ler em voz alta

Apontar cpu.max em C1 e C2, depois memória, swap e tarefas. Ler os valores como microssegundos por período.

Próxima ação

Trocar para o slide 6 na passagem para Sibele.

FALA - Marcos Vinícius Dantas Aguiar (04:00-05:00)

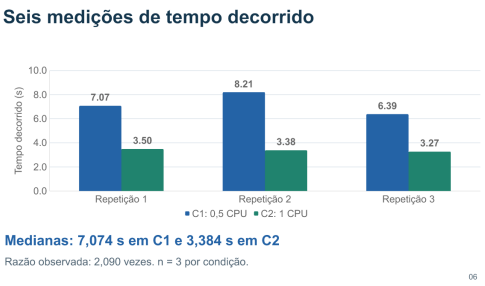
Eu sou Marcos. C1 recebeu quota de meia CPU, e C2, quota de uma CPU. Confirmamos esses limites lendo os arquivos do cgroup antes e depois do trabalho, além de registrar a configuração do Docker.

Em C1, cpu ponto max mostrou cinquenta mil por cem mil microssegundos. Isso significa até cinquenta milissegundos de tempo agregado de CPU por período de cem milissegundos. A quota não reserva um núcleo nem define afinidade.

As duas condições mantiveram duzentos e cinquenta e seis mebibytes de memória, swap adicional zero e limite de sessenta e quatro tarefas, incluindo threads. Também conferimos os arquivos de memória e tarefas. Não provocamos esgotamento de recursos. Sibele apresenta agora os tempos medidos.

Slide 06 - Seis medições de tempo decorrido

Janela 05:00-06:20 | Duração-alvo 01:20



EM TELA - não ler em voz alta

Apontar azul para C1, verde para C2, os três pares e as medianas abaixo do gráfico.

Próxima ação

Trocar para o slide 7; Marcos começa e Sibele continua sem trocar novamente.

FALA - Sibele Oliveira Cruz (05:00-06:20)

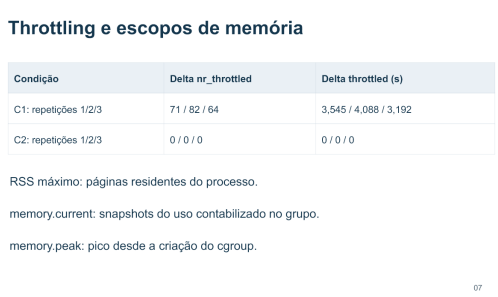
Eu sou Sibele. Cada par de barras representa uma repetição, com azul para meia CPU e verde para uma CPU. As seis execuções fizeram o mesmo trabalho e passaram na verificação de correção. Medimos o tempo do laço, excluindo a criação do contêiner e a inicialização do Python.

A mediana foi aproximadamente sete vírgula zero sete quatro segundos em C1 e três vírgula três oito quatro segundos em C2. A razão entre essas medianas foi dois vírgula zero nove. Isso mostra uma redução observada do tempo quando aumentamos a quota neste ambiente.

Também precisamos olhar a variação. C1 apresentou tempos de cerca de seis vírgula trinta e nove a oito vírgula vinte e um segundos; C2 ficou entre três vírgula vinte e sete e três vírgula cinquenta. Temos apenas três repetições por condição, então esse resultado não demonstra um ganho universal de duas vezes. Marcos vai relacionar esses tempos aos contadores de throttling.

Slide 07 - Throttling e escopos de memória

Janela 06:20-07:20 | Duração-alvo 01:00



EM TELA - não ler em voz alta

Marcos aponta a tabela superior; Sibele assume ao final da passagem e aponta as três definições abaixo. Manter o slide.

Próxima ação

Trocar para o slide 8 após Sibele anunciar Silas.

FALA - Marcos Vinícius Dantas Aguiar (06:20-06:50)

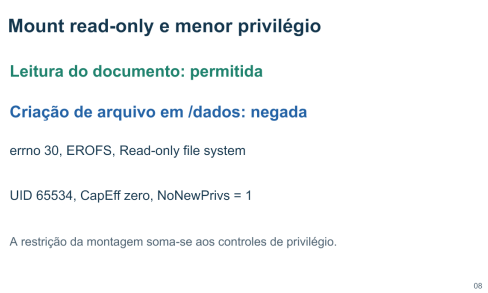
Em C1, o contador de períodos com throttling aumentou nas três execuções: setenta e um, oitenta e dois e sessenta e quatro. Também houve aumento do tempo de throttling. Em C2, esses deltas foram zero na janela observada. Junto com cpu ponto max e com os tempos, isso sustenta que a quota restringiu a carga em C1. Sibele explica agora os escopos da memória.

FALA - Sibele Oliveira Cruz (06:50-07:20)

RSS máximo descreve páginas residentes do processo. Memory current mostra o uso contabilizado no grupo em instantes de coleta. Memory peak registra o pico desde a criação do cgroup, incluindo a inicialização. Essas medidas têm escopos diferentes e não são todas uma média de RAM. O limite foi confirmado sem provocar OOM. Silas vai apresentar a restrição de escrita.

Slide 08 - Mount read-only e menor privilégio

Janela 07:20-08:30 | Duração-alvo 01:10



EM TELA - não ler em voz alta

Apontar leitura permitida, criação negada, EROFS e os controles de privilégio. Não tentar escrever em outro diretório do host.

Próxima ação

Trocar para o slide 9 quando Silas passar a palavra a Christian.

FALA - Silas Santos da Silva (07:20-08:30)

Eu sou Silas. Criamos um documento sintético e montamos seu diretório em barra dados com a opção somente leitura. O contêiner conseguiu ler o documento, mas a tentativa controlada de criar um arquivo nesse mesmo diretório foi negada. O erro foi trinta, EROFS, que indica sistema de arquivos somente leitura.

Também observamos a opção ro no ponto de montagem. A execução usou usuário não root, capabilities efetivas zeradas e a restrição de novos privilégios ativada. Cada mecanismo tem um papel: o mount namespace define a visão das montagens, a opção ro restringe escrita e a identidade limita ações permitidas.

Essa negação comprova a restrição daquele mount, sem demonstrar segurança absoluta. O teste ficou limitado ao diretório sintético. Christian vai mostrar como auditamos as evidências preservadas.

Slide 09 - Evidência selecionada

Janela 08:30-09:30 | Duração-alvo 01:00



EM TELA - não ler em voz alta

0-10 s: explicar no slide. 10-30 s: terminal preparado, executar o auditor e apontar o resultado. 30-50 s: mostrar C1-1.json já aberto. 50-60 s: voltar e passar a palavra. Comandos na página de operação.

Próxima ação

Retornar aos slides e avançar para o 10; Iasmin assume.

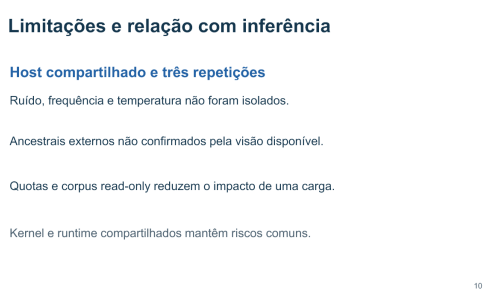
FALA - Christian Will Silva Santos Nunes (08:30-09:30)

Vou mostrar uma evidência verificável. Este comando audita o lote já coletado; ele não repete os experimentos. O resultado informa seis execuções principais válidas e sessenta e oito arquivos conferidos.

No registro C1 um, podemos localizar cpu ponto max com a quota de meia CPU e os deltas dos contadores. O auditor também recalcula o checksum e confere limites, ordem das execuções e limpeza. Assim, a interpretação pode ser confrontada com os arquivos, além dos slides. Iasmin e Silas encerram discutindo os limites do estudo.

Slide 10 - Limitações e relação com inferência

Janela 09:30-10:30 | Duração-alvo 01:00



EM TELA - não ler em voz alta

Iasmin apresenta as três primeiras linhas; Silas continua com a relação com servidores e o risco compartilhado. Manter o slide entre os locutores.

Próxima ação

Trocar para o slide 11 após Silas passar a palavra a Bruno.

FALA - Iasmin Victoria Teixeira Barreto (09:30-10:00)

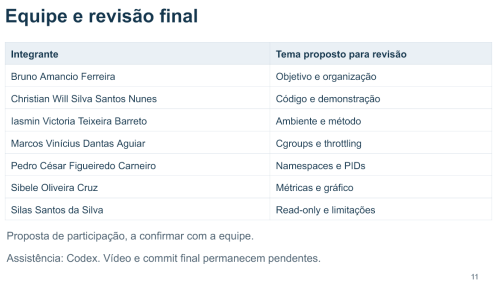
Nossa principal ameaça à validade é o host compartilhado. Fizemos três repetições por condição, sem isolar carga externa, frequência ou temperatura. Além disso, a visão de proc acessível via WSL não permitiu localizar os ancestrais reais dos contêineres. Por isso, os limites externos permanecem não confirmados. As leituras dos limites locais continuam registradas, mas a generalização precisa respeitar essas condições.

FALA - Silas Santos da Silva (10:00-10:30)

Em um servidor de inferência, quotas podem reduzir o impacto de uma carga sobre as demais, e um corpus somente leitura reduz alterações indevidas nos documentos. Ainda permanecem riscos ligados ao host, ao kernel e ao runtime compartilhados. Nosso teste usou aritmética Python, então não mede desempenho de LLM ou GPU. Bruno encerra retomando os resultados e a participação da equipe.

Slide 11 - Equipe e revisão final

Janela 10:30-11:00 | Duração-alvo 00:30



Equipe e revisão final

Integrante	Tema proposto para revisão
Bruno Amancio Ferreira	Objetivo e organização
Christian Will Silva Santos Nunes	Código e demonstração
Iasmin Victoria Teixeira Barreto	Ambiente e método
Marcos Vinícius Dantas Aguiar	Cgroups e throttling
Pedro César Figueiredo Carneiro	Namespaces e PIDs
Sibele Oliveira Cruz	Métricas e gráfico
Silas Santos da Silva	Read-only e limitações

Proposta de participação, a confirmar com a equipe.

Assistência: Codex. Vídeo e commit final permanecem pendentes.

11

EM TELA - não ler em voz alta

Manter a tabela da equipe. Antes da apresentação final, atualizar a proposta de participação e as pendências do rodapé com informações confirmadas.

Próxima ação

Encerrar a exposição. Abrir o período separado de 3-5 minutos para perguntas.

FALA - Bruno Amancio Ferreira (10:30-11:00)

Para concluir, observamos namespaces distintos, limites efetivos de cgroup e uma restrição real de escrita. Com o mesmo trabalho, a quota maior apresentou menor tempo mediano nesta execução. O repositório reúne código, dados e documentação para conferência. Registramos a assistência do Codex, e a revisão e a responsabilidade acadêmica pertencem à equipe. Obrigado. Estamos disponíveis para as perguntas.

Operação da demonstração

Apoio ao slide 9. Esta página orienta Christian e a pessoa que controla a tela.

Preparar antes de apresentar ou gravar

Abrir o repositório na sua raiz. Usar um ambiente em que python execute a versão configurada. No computador desta equipe, os ensaios anteriores usaram o ambiente Python no Ubuntu WSL.

Conferir o comando antes do ensaio e deixar o terminal com fonte grande. Não instalar programas, baixar imagem ou executar novamente a matriz durante a apresentação.

Deixar dois arquivos já abertos no editor: runs/C1-1.json e mount-readonly.json, nos caminhos completos abaixo. Esses arquivos registram observações anteriores; não são eventos que estão acontecendo agora.

Comando completo em uma única linha

```
python atividade3/scripts/audit.py atividade3/results/raw/20261108T013949Z-19921f7f
```

No Bash, a quebra de linha do slide usa uma barra invertida. A versão acima pode ser colada como um único comando. O auditor só verifica o lote existente; não precisa recriar os contêineres.

O que apontar na tela

1. Saída do auditor

Apontar valid=true, main_runs=6 e verified_files=68. Se a execução atual falhar, explicar o erro e abrir os registros preservados; não afirmar sucesso atual.

2. Registro C1-1

Apontar iterations=20000000, checksum=2589261568, correct=true, cpu.max="50000 100000" e cpu_stat_delta. Em C1-1, nr_throttled aumentou 71 e throttled_usec aumentou 3544785.

3. Mount read-only

Apontar content, write_denied=true, errno=30, uid=65534 e os controles de privilégio. Usar esse arquivo principalmente no vídeo; na apresentação, respeitar o minuto reservado à demonstração.

Arquivos para deixar abertos

`atividade3/results/raw/20261108T013949Z-19921f7f/runs/C1-1.json`

`atividade3/results/raw/20261108T013949Z-19921f7f/mount-readonly.json`

Plano de contingência: se o terminal não responder, mostrar os arquivos locais e explicar que são registros da coleta. Na apresentação, passar ao slide 10 sem consumir o tempo com diagnóstico. No vídeo, interromper o trecho, corrigir o ambiente e gravar novamente, preservando a honestidade sobre o que foi executado.

Apoio para as perguntas

Reservar 3-5 minutos após a exposição. As respostas abaixo são um apoio de estudo, não falas obrigatórias.

Iasmin - Onde está o kernel compartilhado?

No ambiente Linux intermediário da infraestrutura WSL2 em que executa o daemon. As imagens fornecem o espaço de usuário; cada contêiner não inicia seu próprio kernel.

Pedro - Como dois processos podem ter PID 1?

A numeração é relativa a cada PID namespace. Na visão informada pelo daemon, os processos tinham PIDs diferentes: 567 e 612.

Marcos - Meia CPU significa meio núcleo reservado?

É uma quota de tempo agregado por período. Não estabelece afinidade nem reserva de núcleo. Em C1, cpu.max mostrou 50000 100000 microssegundos.

Christian - Como sabemos que fizeram o mesmo trabalho?

N ficou congelado em vinte milhões de iterações. Todas concluíram o laço e concordaram com o checksum calculado por fórmula independente.

Sibele - Por que a razão não foi exatamente dois?

Há três observações por condição e interferência não isolada do host. A razão de medianas 2,090 é o resultado observado, sem provar um ganho universal.

Sibele - Por que RSS difere de memory.current?

São escopos e contabilizações distintos. RSS é do processo; memory.current é do grupo nos instantes coletados; memory.peak cobre a vida do cgroup.

Silas - EROFS comprova segurança absoluta?

Comprova a restrição daquele mount, junto com a opção ro observada. Kernel, runtime e host continuam compartilhados e exigem proteção e atualização.

Iasmin - Os ancestrais estavam sem limite?

Não foi possível confirmá-los pela visão de proc disponível. A ausência de acesso foi registrada como limitação, sem transformá-la em um valor ilimitado.

Bruno pode explicar a organização e a declaração de IA. A equipe deve distinguir apoio automatizado de contribuição efetivamente realizada e não inventar o identificador do modelo quando não estiver verificado.